Table des matières

Cours 06 — Agrégation avancée et pivots 1
Objectifs 1
Théorie 1
Points clés à retenir 9
Exercices 10

Cours 06 — Agrégation avancée et pivots

Module 6/12 — Formation SQL avancé

Comment lire ce module — C'est le SQL du reporting, celui qui produit un tableau de bord complet (le détail, ses sous-totaux et le total général) en une seule requête, puis qui fait pivoter les lignes en colonnes pour obtenir un vrai tableau croisé. On part du `GROUP BY` que vous maîtrisez et on lui ajoute les extensions que les outils de BI utilisent en coulisse. La théorie est ici ; les deux laboratoires (`Labo_06_OLAP_SQL.md` et `Labo_06_OLTP_SQL.md`) vous font tout taper.

Où s'arrêter — Ce cours fait environ neuf heures et demie en tout, près de quatre séances de deux heures et demie. La théorie et les deux labos en prennent un peu plus d'une, les exercices et leur corrigé le reste. La coupure tombe après la section 5 : les sections 1 à 5 font les sous-totaux en une passe, les sections 6 à 8 font tenir plusieurs mesures, un tableau croisé ou une liste dans une seule ligne. Les labos se coupent au même endroit, après leur sous-partie 2.2.

Objectifs

À la fin de ce module, vous saurez :

- produire des sous-totaux et un total général en une passe avec `ROLLUP`, `CUBE` et `GROUPING SETS` ;
- distinguer un `NULL` de donnée d'un `NULL` de sous-total avec la fonction `GROUPING()` ;
- écrire des agrégats conditionnels (`SUM(CASE WHEN …)`) et connaître le raccourci `FILTER` ;
- construire un tableau croisé avec `PIVOT`, et faire l'inverse avec `UNPIVOT` ;
- recoller les valeurs d'un groupe en texte ou en liste (`STRING_AGG`, `LIST`, `ARRAY_AGG`), sans répétition ni désordre ;
- savoir ce qui est standard et ce qui est propre à un moteur (`FILTER`, `PIVOT`, `DISTINCT ON`).

Théorie

© 2026 iAlexMG. Tous droits réservés.

1. Le problème : le détail et ses totaux, d'un seul coup

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, partie 1.

Un `GROUP BY` classique donne un niveau de regroupement, les ventes par année et par pays par exemple. Mais un rapport digne de ce nom veut plusieurs niveaux à la fois : le détail par pays, le sous-total de chaque année, et le total général tout en bas.

La solution naïve, c'est d'écrire trois requêtes et de les coller avec `UNION ALL`. C'est long, et facile à désynchroniser. SQL offre mieux : des extensions du `GROUP BY` qui calculent tous ces niveaux en une seule passe. Trois mots-clés, `GROUPING SETS`, `ROLLUP`, `CUBE`.

2. `GROUPING SETS` : choisir précisément les regroupements

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.1.

`GROUPING SETS` liste explicitement les regroupements voulus. Chaque parenthèse est un niveau ; `()`, parenthèses vides, désigne le total général.

```
SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY GROUPING SETS (
    (d.CalendarYear, t.SalesTerritoryCountry),    -- le detail : annee x pays
    (d.CalendarYear),                             -- sous-total par annee
    ()                                            -- total general
);
```

Dans les lignes de sous-total, la colonne non regroupée ressort à `NULL` : la ligne du sous-total 2013 a `pays = NULL`, la ligne du total général a `annee` et `pays` tous les deux à `NULL`. On apprend à étiqueter ces `NULL` en section 5.

3. `ROLLUP` : les sous-totaux en cascade

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, partie 1.

`ROLLUP` est le raccourci pour une hiérarchie de sous-totaux, du plus fin au plus grossier. `ROLLUP (a, b)` calcule `(a, b)`, puis `(a)`, puis `()`. C'est exactement le `GROUPING SETS` de la section 2, en une ligne :

```
SELECT d.CalendarYear AS annee, t.SalesTerritoryCountry AS pays,
       SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY ROLLUP (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY annee, pays;
```


© 2026 iAlexMG. Tous droits réservés.

C'est l'outil du détail avec ses sous-totaux et son total, quand les colonnes forment une hiérarchie naturelle (année puis pays, ou catégorie puis sous-catégorie). L'ordre compte : `ROLLUP (annee, pays)` fait des sous-totaux par année, `ROLLUP (pays, annee)` les ferait par pays.

Une conséquence est à poser tout de suite, parce qu'elle vaut pour les trois extensions et qu'elle explique la moitié des accidents du cours : un résultat de `ROLLUP` mélange des grains. Les lignes de détail et les lignes de synthèse habitent le même tableau, et rien dans le type des colonnes ne les distingue. C'est ce qu'on demande à un rapport destiné à l'affichage. C'est aussi ce qui rend ce résultat impropre à tout traitement en aval : sommer la colonne `ventes` du `ROLLUP` ci-dessus rend 88 076 031,66 $, exactement trois fois les 29 358 677,22 $ de l'entrepôt, puisque chaque vente y figure au détail, dans son sous-total d'année et dans le total général. Une moyenne n'a pas plus de sens, et une jointure sur `pays` laisse tomber les sous-totaux. La règle qui en découle est simple : un `ROLLUP` termine une chaîne, il ne l'alimente pas.

4. `CUBE` : toutes les combinaisons

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.1.

`CUBE` va plus loin : il calcule toutes les combinaisons possibles des colonnes. `CUBE (a, b)` produit `(a, b)`, `(a)`, `(b)` et `()`, y compris le sous-total par pays toutes années confondues, que `ROLLUP` ne donnait pas.

```
  GROUP BY CUBE (d.CalendarYear, t.SalesTerritoryCountry)
```

À réserver quand on veut croiser dans tous les sens. Avec n colonnes, `CUBE` génère 2 puissance n niveaux : puissant, mais vite coûteux. Ne mettez dans le `CUBE` que les axes qui vous intéressent vraiment.

L'image à retenir — `GROUPING SETS`, c'est le menu à la carte : vous listez chaque niveau. `ROLLUP`, c'est le menu en cascade : une hiérarchie du fin au grossier. `CUBE`, c'est le buffet : toutes les combinaisons.

5. `GROUPING()` : reconnaître les lignes de sous-total

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.2.

Un piège attend. Comment distinguer une ligne où `pays = NULL` parce que c'est un sous-total d'une ligne où `pays` est réellement inconnu dans les données ? La fonction `GROUPING(colonne)` répond : elle renvoie 1 si la colonne est agrégée sur cette ligne, donc s'il s'agit d'un sous-total, et 0 sinon. On s'en sert pour poser de vrais libellés :

```
  SELECT
      CASE WHEN GROUPING(d.CalendarYear) = 1 THEN 'Toutes années'
           ELSE CAST(d.CalendarYear AS VARCHAR) END AS annee,
      CASE WHEN GROUPING(t.SalesTerritoryCountry) = 1 THEN 'Tous pays'
           ELSE t.SalesTerritoryCountry END AS pays,
      SUM(f.SalesAmount) AS ventes
  FROM dbo.FactInternetSales AS f
  JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
  JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
```


© 2026 iAlexMG. Tous droits réservés.

```sql
GROUP BY ROLLUP (d.CalendarYear, t.SalesTerritoryCountry)
ORDER BY GROUPING(d.CalendarYear), d.CalendarYear,
         GROUPING(t.SalesTerritoryCountry), t.SalesTerritoryCountry;
```

Les sous-totaux affichent désormais « Toutes années » et « Tous pays » au lieu d'un `NULL` ambigu, et le rapport est lisible tel quel.

Rappel — `CASE` est présenté au Cours 02, section 3, et `CAST` à la section 4 du même cours.

Piège — La tentation est grande d'écrire `COALESCE(pays, 'Total')`, qui est plus court. Ça marche tant qu'aucun pays n'est réellement nul dans les données, et ça produit deux lignes intitulées « Total » le jour où il y en a un. `GROUPING()` répond à la question « cette ligne est-elle un sous-total » ; `COALESCE` répond à « cette donnée est-elle manquante ». Ce sont deux questions différentes.

6. Agrégats conditionnels : compter et sommer sous condition

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.3.

Souvent, on veut plusieurs mesures côte à côte, chacune sur un sous-ensemble différent. L'astuce consiste à glisser un `CASE` dans la fonction d'agrégat. Le `CASE` ne garde que les lignes voulues ; les autres deviennent `NULL`, que `SUM` et `COUNT` ignorent.

Les ventes par année, une colonne par continent :

```sql
SELECT d.CalendarYear AS annee,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'North America' THEN f.SalesAmount END) AS amerique_nord,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Europe'        THEN f.SalesAmount END) AS europe,
       SUM(CASE WHEN t.SalesTerritoryGroup = 'Pacific'       THEN f.SalesAmount END) AS pacifique
FROM dbo.FactInternetSales AS f
JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
GROUP BY d.CalendarYear
ORDER BY annee;
```

Une année par ligne, trois continents en colonnes : c'est déjà un tableau croisé, fabriqué à la main. Le même motif avec `COUNT(CASE WHEN … THEN 1 END)` compte combien de lignes remplissent telle condition.

Notez l'absence de `ELSE`. Ce n'est pas un oubli, et la raison n'est pas celle qu'on récite d'habitude. On dit souvent qu'`ELSE 0` ne changerait rien à un `SUM`, puisque ajouter des zéros ne modifie pas un total. C'est vrai du montant, et faux du tableau : dans un groupe où aucune ligne ne remplit la condition, `SUM(CASE WHEN … THEN mesure END)` rend une case vide et `SUM(CASE WHEN … THEN mesure ELSE 0 END)` rend un zéro. Mesuré sur nos deux moteurs. Or une case vide et un zéro ne disent pas la même chose : l'une dit « ce pays n'a rien vendu cette année-là », l'autre dit « il a vendu, pour zéro dollar ». Dans un tableau croisé destiné à être lu, cette distinction est la première chose que le lecteur regarde.

Le tort du `ELSE 0` est plus net encore sur les autres fonctions. Il ruine un `AVG`, où les zéros deviennent des valeurs comptées et tirent la moyenne vers le bas, et un `COUNT`, qui compte alors toutes les lignes du groupe : sur l'entrepôt, les ventes allemandes de 2011 se comptent à 175 sans `ELSE` et à 2 216 avec, soit toutes les ventes de l'année. On omet donc le `ELSE` par principe, et pas seulement quand la fonction du moment y est sensible. Si vous voulez un zéro à l'affichage,

© 2026 iAlexMG. Tous droits réservés.

posez-le à l'extérieur, par `COALESCE(SUM(CASE WHEN … THEN mesure END), 0)` : le calcul reste juste, et le zéro redevient ce qu'il est, une décision de présentation.

Portabilité — Il existe une écriture plus lisible, la clause `FILTER` : `SUM(f.SalesAmount) FILTER (WHERE t.SalesTerritoryGroup = 'Europe')`. Elle appartient à la norme SQL, et nos deux terrains l'acceptent, au même résultat près : 8 930 042 $ pour l'Europe, par `FILTER` comme par `SUM(CASE …)`. Elle n'est pas répandue partout pour autant, et notamment Snowflake ne la connaît pas d'après sa documentation. Le `SUM(CASE …)`, lui, fonctionne partout et reste la valeur sûre pour tout code appelé à voyager. Écrivez `FILTER` quand le code reste chez vous, `SUM(CASE …)` quand il part.

7. `PIVOT` et `UNPIVOT` : le tableau croisé, en clair

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.4.

Le `SUM(CASE …)` de la section 6 marche, mais il faut répéter une colonne par valeur. `PIVOT` fait la même chose avec une syntaxe dédiée : on nomme la mesure, la colonne à éclater en colonnes, et la liste des valeurs.

```sql
SELECT *
FROM (
    SELECT d.CalendarYear AS annee, t.SalesTerritoryGroup AS continent,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
) AS src
PIVOT (SUM(montant) FOR continent IN ('North America', 'Europe', 'Pacific')) AS p
ORDER BY annee;
```

Deux règles à connaître. La sous-requête `src` ne doit contenir que les colonnes utiles (l'axe des lignes, l'axe à pivoter, la mesure) : toute colonne en trop entre dans le regroupement implicite et change le grain du résultat, sans lever la moindre erreur. Et les valeurs, `'North America'` et les autres, sont écrites en dur : sous cette forme, `PIVOT` ne devine pas la liste des colonnes, et il ignore silencieusement toute valeur qui n'y figure pas.

Cette seconde règle mérite une nuance, parce que notre entrepôt offre une porte de sortie que la forme standard n'a pas. DuckDB accepte une écriture qui lui est propre, où la liste des valeurs disparaît :

```sql
PIVOT (
    SELECT d.CalendarYear AS annee, t.SalesTerritoryGroup AS continent,
           f.SalesAmount AS montant
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    JOIN dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = f.SalesTerritoryKey
) ON continent USING SUM(montant)
ORDER BY annee;
```


© 2026 iAlexMG. Tous droits réservés.

Le moteur lit les valeurs distinctes de `continent` et fabrique une colonne pour chacune. Les deux écritures rendent ici le même tableau, aux colonnes près, qui sortent dans l'ordre alphabétique plutôt que dans celui de la liste. C'est plus court, et surtout ça ne se périme pas : le jour où un quatrième continent apparaît, la première écriture l'oublie en silence et la seconde l'ajoute.

Le prix à payer est double, et il faut le connaître avant de préférer la seconde. D'abord elle ne voyage pas : c'est une extension du moteur, absente de la norme. Ensuite le résultat devient dépendant des données, ce qui est gênant quand un rapport doit avoir des colonnes stables d'un mois à l'autre. Notre entrepôt le montre bien : `DimSalesTerritory` compte quatre groupes, Europe, North America, Pacific et un « NA » du genre de ceux du Cours 01. Le pivot automatique n'en rend que trois, parce que le groupe « NA » n'a aucune vente. Une valeur qui n'existe pas dans les données n'existe pas comme colonne, et le tableau change de forme le jour où une première vente y tombe.

`UNPIVOT` fait l'inverse : il replie des colonnes en lignes, `continent` et `montant`. C'est précieux pour normaliser un tableau large, par exemple des colonnes `2011`, `2012`, `2013` qu'on veut ramener à deux colonnes `annee` et `ventes`, plus faciles à agréger. C'est la forme sous laquelle une donnée arrive quand elle vient d'un tableur, et c'est donc la porte d'entrée de beaucoup de chargements.

Il a un piège, et c'est le même mécanisme que le `ELSE` de la section 6, vu par l'autre bout : les cases vides ne donnent pas de ligne. Un tableau de cinq années sur quatre pays a vingt cases ; s'il en a une vide, le repli rend dix-neuf lignes, sans erreur ni avertissement. Le labo le mesure sur l'Allemagne, qui n'a aucune vente en 2010. Tant qu'on replie pour ré-agréger, cette disparition ne change rien, puisqu'une somme ignore de toute façon les valeurs nulles. Elle compte dès qu'on compte, dès qu'on joint, ou dès qu'on veut prouver qu'un tableau reçu est complet. La parade sur l'entrepôt est un `COALESCE` par colonne repliée, qui remplace le trou par un zéro, et qui vous oblige donc à décider ce que le trou voulait dire.

Portabilité — Ce comportement est le même sur Snowflake d'après sa documentation, qui écarte les valeurs nulles par défaut, mais qui offre en plus `EXCLUDE NULLS` et `INCLUDE NULLS` pour l'écrire dans la requête. DuckDB ne connaît ni l'un ni l'autre, mesuré : les écrire rend une erreur de syntaxe. PostgreSQL, lui, n'a pas d'`UNPIVOT` du tout ; on y replie par une jointure latérale sur une liste de valeurs, qui garde les cases vides. Des trois comportements, c'est le seul qui ne fasse rien disparaître sans qu'on le demande.

8. Agréger en texte ou en liste : `STRING_AGG`, `LIST` et `ARRAY_AGG`

Labo : `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`, sous-partie 2.5.

Un agrégat ne rend pas toujours un nombre. `STRING_AGG(colonne, séparateur)` recolle en un seul texte les valeurs du groupe, et c'est la façon de dire « lesquelles » là où `COUNT` dit « combien ». Les sous-catégories de chaque catégorie, en une ligne par catégorie :

```sql
SELECT c.EnglishProductCategoryName AS categorie,
       STRING_AGG(s.EnglishProductSubcategoryName, ', '
                  ORDER BY s.EnglishProductSubcategoryName) AS sous_categories
FROM dbo.DimProductCategory    AS c
JOIN dbo.DimProductSubcategory AS s ON s.ProductCategoryKey = c.ProductCategoryKey
GROUP BY c.EnglishProductCategoryName
ORDER BY categorie;
```


© 2026 iAlexMG. Tous droits réservés.

categorie	sous_categories
Accessories	Bike Racks, Bike Stands, Bottles and Cages, Cleaners, Fenders, Helmets, Hydration Packs, Lights, Locks, Panniers, Pumps, Tires and Tubes
Bikes	Mountain Bikes, Road Bikes, Touring Bikes
Clothing	Bib-Shorts, Caps, Gloves, Jerseys, Shorts, Socks, Tights, Vests
Components	Bottom Brackets, Brakes, Chains, Cranksets, Derailleurs, Forks, Handlebars, Headsets, Mountain Frames, Pedals, Road Frames, Saddles, Touring Frames, Wheels

Le `ORDER BY` se place à l'intérieur de l'agrégat, et il n'est pas décoratif : sans lui, la norme ne garantit aucun ordre, et la même liste peut sortir autrement d'une exécution à l'autre, ce qui suffit à faire échouer une comparaison de texte. Comme les autres agrégats, `STRING_AGG` saute les `NULL`, mesuré sur nos deux terrains.

Deux pièges guettent dès qu'on agrège en texte à travers une table de faits. Le premier est la répétition. Les couleurs des vélos vendus en ligne, prises sur les ventes, reviennent une fois par ligne de vente : 15 205 éléments dans la liste, pour cinq couleurs. `DISTINCT` les réduit, et les couleurs vendues de chaque catégorie deviennent lisibles :

```
SELECT c.EnglishProductCategoryName                    AS categorie,
       STRING_AGG(DISTINCT p.Color, ', ' ORDER BY p.Color) AS couleurs_vendues
FROM dbo.FactInternetSales     AS f
JOIN dbo.DimProduct            AS p ON p.ProductKey = f.ProductKey
JOIN dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
JOIN dbo.DimProductCategory    AS c ON c.ProductCategoryKey = s.ProductCategoryKey
GROUP BY c.EnglishProductCategoryName
ORDER BY categorie;
```

Trois lignes : Accessories (Black, Blue, NA, Red, Silver), Bikes (Black, Blue, Red, Silver, Yellow), Clothing (Black, Blue, Multi, White, Yellow). Les composants n'ont aucune vente en ligne, et la jointure interne les a écartés. Avec `DISTINCT`, le tri doit porter sur la valeur agrégée elle-même : `ORDER BY p.EnglishProductName` y est refusé par les deux moteurs, au même message près (`In a DISTINCT aggregate, ORDER BY expressions must appear in the argument list`), parce qu'une couleur portée par dix produits n'a pas de nom de produit à elle.

Le second piège est la taille. Un `STRING_AGG` sans `GROUP BY` sur les 60 398 ventes rend un seul texte de 330 382 caractères : rien ne plante, et le résultat est inutilisable. Un texte agrégé se destine à un humain qui le lit ; au-delà d'une ligne d'écran, il faut regrouper plus fin ou compter.

Quand le résultat doit être retravaillé plutôt que lu, on agrège en liste. `LIST` sur DuckDB, et `ARRAY_AGG` sur les deux moteurs, rendent une valeur de type tableau, `VARCHAR[]` sur l'entrepôt, dont chaque élément reste accessible, compté, déplié en lignes. Les tableaux et leur dépliage sont au Cours 08, et le Cours 09 s'en sert pour tirer la première et la dernière valeur d'une période.

© 2026 iAlexMG. Tous droits réservés.

Portabilité — `STRING_AGG` et `ARRAY_AGG` s'écrivent à l'identique sur nos deux terrains, avec `DISTINCT` et `ORDER BY` à l'intérieur, mesuré. La norme écrit `LISTAGG`, que DuckDB accepte aussi et que PostgreSQL refuse (« la fonction listagg n'existe pas »), mesuré ; Snowflake emploie `LISTAGG` et `ARRAY_AGG`, d'après sa documentation. `STRING_AGG` est donc la forme la plus sûre entre nos deux terrains, et `LISTAGG` celle à chercher d'abord sur un moteur inconnu.

Portabilité — Le bilan de ce cours tient en trois lignes, et il est plus favorable qu'on ne le croirait. Les extensions du `GROUP BY`, soit `GROUPING SETS`, `ROLLUP`, `CUBE` et `GROUPING()`, sont du standard, et tournent à l'identique sur nos deux terrains, mesuré ; Snowflake les a aussi, d'après sa documentation. Trois écritures de confort, `FILTER`, `DISTINCT ON` et `STRING_AGG`, passent elles aussi des deux côtés, alors qu'on les croit souvent propres à PostgreSQL. Ne reste qu'un seul vrai écart, `PIVOT` et `UNPIVOT`, que l'entrepôt accepte et que PostgreSQL refuse par « erreur de syntaxe » : c'est là, et là seulement, qu'il faut redescendre au `SUM(CASE …)`. Pour obtenir une ligne par groupe, les deux terrains ont `DISTINCT ON`, et l'entrepôt ajoute le réflexe du Cours 04, `QUALIFY ROW_NUMBER() OVER (…) = 1`.

9. Sur les deux terrains : OLTP et OLAP

Labo : `Labo_06_OLTP_SQL.md`, partie 3.

Ce cours contient à la fois le morceau le plus portable de la formation et le moins portable de tous. Les extensions du `GROUP BY` sont du standard bien implanté et s'écrivent à l'identique des deux côtés, comme les deux écritures de confort qu'on croit souvent propres à PostgreSQL, `FILTER` et `DISTINCT ON`. Le pivot, lui, n'existe pas du tout sur PostgreSQL, et c'est le seul écart de syntaxe que ce cours laisse debout.

Ce qu'on veut	Voie OLTP (PostgreSQL, schéma normalisé)	Voie OLAP (DuckDB, étoile)
L'année d'une vente	`EXTRACT` sur `orderdate`, une expression et non une colonne	`CalendarYear`, une vraie colonne
Le nom d'un pays	un code sur deux lettres, à joindre au référentiel	en toutes lettres dans la dimension
Le continent	la colonne `group`, un mot réservé, à écrire `t."group"`	`SalesTerritoryGroup`, sans histoire
Sous-totaux et total	`ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`	identiques, au caractère près
Agrégat conditionnel	`SUM(CASE …)`, ou la clause `FILTER`	les deux aussi
Une ligne par groupe	`DISTINCT ON`	`DISTINCT ON` aussi, ou `QUALIFY ROW_NUMBER() … = 1`
Tableau croisé	aucun `PIVOT` : agrégat conditionnel, `crosstab`, ou format long	`PIVOT` et `UNPIVOT`, plus une forme propre au moteur
Agréger en texte ou en liste	`STRING_AGG`, `ARRAY_AGG` ; pas de `LISTAGG`	`STRING_AGG`, `LIST`, `ARRAY_AGG` et `LISTAGG`

Deux remarques valent d'être retenues au-delà du tableau.

© 2026 iAlexMG. Tous droits réservés.

La première porte sur l'alias. Sur le transactionnel, l'année est une expression, pas une colonne. PostgreSQL accepte qu'on la désigne dans le `ROLLUP` par son alias ou par sa position, comme dans tout `GROUP BY` (Cours 02, section 7), à une condition : que l'alias ne reprenne pas le nom d'une colonne des tables lues, sans quoi c'est la colonne qui l'emporte. Cette tolérance est une extension, pas une règle du langage : un moteur qui suit la norme à la lettre refuse l'alias comme la position, l'un et l'autre naissant après le `GROUP BY`. La CTE du Cours 05 règle la question partout : à l'intérieur de la requête qui la consomme, l'année n'est plus un alias mais une véritable colonne. Les cours se répondent, et c'est voulu.

La seconde porte sur l'absence de `PIVOT`, qui n'est pas un manque de PostgreSQL mais une conséquence de sa rigueur : le nombre et le type des colonnes d'un résultat doivent être connus avant l'exécution, or un pivot dont les colonnes viennent des données ne les connaît qu'après.

La forme automatique de la section 7 en est la démonstration involontaire. Quand on écrit `PIVOT (…) ON continent USING SUM(montant)`, le moteur doit lire les données pour savoir combien de colonnes il va rendre, et il en rend trois parce que le groupe « NA » n'a pas de vente. Rien n'interdit qu'il en rende quatre demain. C'est commode et c'est précisément ce que PostgreSQL refuse de faire. Les deux positions se défendent, et il vaut mieux comprendre le désaccord que trancher : l'un privilégie le confort d'écriture, l'autre la stabilité du contrat de sortie.

La bonne réponse n'est donc pas de chercher un équivalent à tout prix, c'est de se demander si le pivot a sa place dans la requête. Livrer les données au format long, une ligne par combinaison d'axes, et laisser l'outil d'affichage croiser, c'est une décision d'architecture qui règle la question pour de bon et qui résiste à l'apparition d'une valeur nouvelle.

Le même travail, deux fois — Le Cours 06 a deux laboratoires, `Labo_06_OLTP_SQL.md` et `Labo_06_OLAP_SQL.md`. Le labo transactionnel se termine sur les trois voies possibles quand `PIVOT` n'existe pas. C'est le passage le plus utile du cours pour quiconque travaillera un jour sur plusieurs moteurs.

Points clés à retenir

- Un `GROUP BY` classique donne un seul niveau. Pour le détail, ses sous-totaux et le total en une passe : `GROUPING SETS` (à la carte), `ROLLUP` (cascade hiérarchique), `CUBE` (toutes les combinaisons, 2 puissance n niveaux).
- Dans les lignes de sous-total, la colonne agrégée vaut `NULL` ; `GROUPING(col)` vaut 1 sur ces lignes, de quoi poser des libellés « Toutes années » ou « Total ». Le `COALESCE` ne remplace pas `GROUPING()`, il répond à une autre question.
- Un résultat de `ROLLUP` mélange des grains : il est fait pour être affiché, pas pour être ré-agrégé en aval, où il compte chaque vente trois fois (section 3).
- Agrégat conditionnel : `SUM(CASE WHEN cond THEN mesure END)` calcule une mesure sur un sous-ensemble, et en aligner plusieurs fabrique un tableau croisé à la main. Jamais de `ELSE 0` : il ne change pas le total, mais il change une case vide en zéro, et il fausse `AVG` et `COUNT`.
- `PIVOT` éclate une colonne de valeurs en colonnes, `UNPIVOT` replie des colonnes en lignes. Sous la forme standard, les valeurs pivotées s'écrivent en dur, et celles qui manquent à la liste sont ignorées sans erreur. L'entrepôt offre en plus une forme qui les découvre, au prix d'un nombre de colonnes qui dépend des données. Au repli, une case vide ne donne pas de ligne : c'est le piège silencieux d'`UNPIVOT`, que seul un `COALESCE` par colonne désamorce.
- `STRING_AGG(col, ', ' ORDER BY col)` dit « lesquelles » là où `COUNT` dit « combien ». Sans `ORDER BY`, l'ordre n'est pas garanti ; à travers une table de faits, sans `DISTINCT`, chaque valeur revient une fois par ligne. `LIST` et `ARRAY_AGG` rendent un tableau qu'on retravaille.

© 2026 iAlexMG. Tous droits réservés.

- Portabilité : `ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUPING()`, `FILTER`, `DISTINCT ON` et `STRING_AGG` passent sur les deux terrains. Le seul écart de syntaxe est le pivot, avec `QUALIFY` : l'entrepôt les a, PostgreSQL non. Le repli sûr est toujours `SUM(CASE …)`.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_06_SQL.md`. Faites-les après les deux laboratoires : plusieurs énoncés supposent que les manipulations ont été faites des deux côtés.

© 2026 iAlexMG. Tous droits réservés.